

CSDS HW - Week 7

Question 1) Let's make an automated recommendation system for the PicCollage mobile app.

Download the file `piccollage_accounts_bundles.csv` from Canvas. You may use `read.csv()` to load the file or you can optionally try learning how to using `data.table` – a high performance package for reading, writing, and managing large data sets. *It will take time to fully learn data.table — but here's some starter code:*

```
library(data.table)
ac_bundles_dt <- fread("piccollage_accounts_bundles.csv")
ac_bundles_matrix <- as.matrix(ac_bundles_dt[, -1, with=FALSE])
```

- a. Let's explore to see if any sticker bundles seem intuitively similar:
 - i. **(recommended)** Download PicCollage onto your mobile from the App Store and take a look at the style and content of various bundles in their Sticker Store (iOS app: can see how many recommendations does each bundle have? Android app might not have recommendations)
 - ii. Find a single sticker bundle that is both in our limited data set and also in the app's Sticker Store (e.g., "sweetmothersday"). Then, use your intuition to recommend (guess!) five other bundles in our dataset that might have *similar* usage patterns as this bundle.
- b. Let's find similar bundles using geometric models of similarity:
 - i. Let's create **cosine-similarity** based recommendations for a given bundle:
 1. Create a `cosine_sim(a, b)` function that returns the cosine similarity of two vectors.
 2. Create a `cosine_recos(items_matrix, item_name)` function that takes a matrix of items and the name of a specific item you are interested in, and returns five (5) recommendations for that item based on cosine similarity. Some hints/suggestions:

```
cosine_recos <- function(items_matrix, item_name) {
  item_vector <- ... # extract column vector for item_name from matrix
  sims <- ...       # apply cosine_sim on all columns versus item_vector
  top_names <- ...   # item names of the sorted cosine similarity values
  return(...) <- ... # return the names of the top five similar items
}
```
 3. What are the top 5 recommendations for the bundle you chose to explore earlier?
 - ii. Let's create **correlation** based recommendations:
 1. Reuse the `cosine_recos(...)` function you created above but give it an items matrix where each item (column) has already been mean-centered in advance.
 2. What are the top 5 recommendations using correlations?
 - iii. Let's create **adjusted-cosine** based recommendations.
 1. Reuse the `cosine_recos(...)` function you created above but give it an items matrix where each observation (row) has already been mean-centered in advance.
 2. What are the top 5 recommendations using adjusted cosine?
- c. *(required, but not graded)* Let's reflect on these three variations of the cosine similarity: What is the conceptual difference in cosine similarity, correlation, and adjusted cosine?
(you may consult others, search online, and/or ask an AI about this)

- d. Let's apply the same cosine similarity idea to a very different dataset: the meaning of book descriptions as encoded in LLM embeddings.

What is an embedding? A sentence embedding is a vector of numbers (in this case, 384 length) that encodes the meaning of a piece of text as a vector in a high-dimensional space. Embedding vectors are learned by an AI language model and reflect its interpretation of the meaning of the text. The individual numbers are not human-interpretable — you cannot point at any dimension (row) and say what it means. But texts with similar embedding vectors tend to have a similar meaning.

best_books_ever.csv — **(recommended)** List of popular books with their title, information, description, etc. You may download this file to see full information. We are interested in their titles and descriptions.

description_embeddings.rds — **(required)** Embedding vectors of the description of all the books. An RDS file is a more efficient way to store R data objects than CSV. Load this file using `readRDS()`. This is a matrix where each column is a book and each row is one of the 384 embedding dimensions.

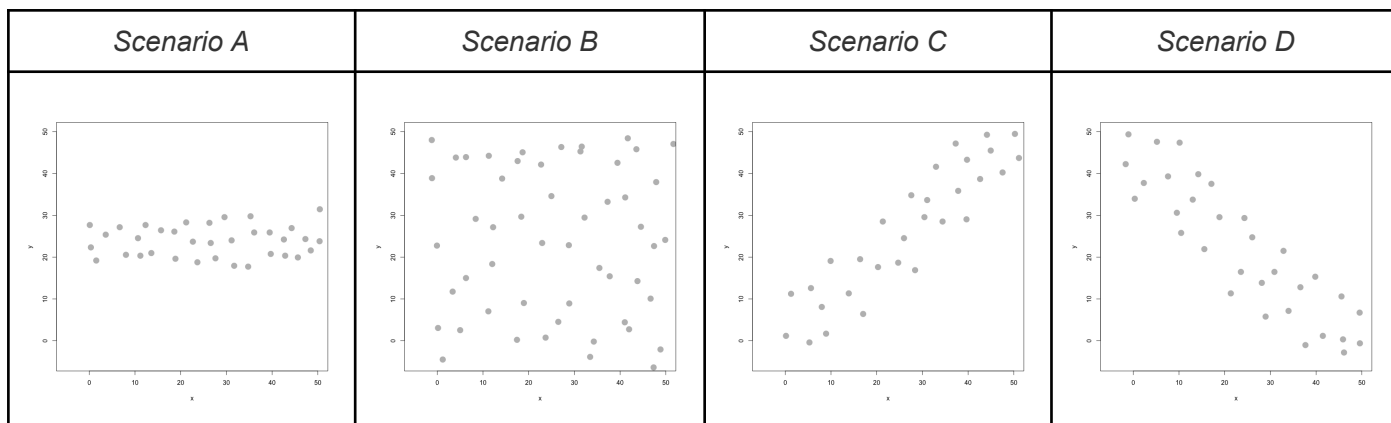
- i. Pick a book that you are either familiar with or has an interesting description. Load the file `description_embeddings.rds` and use your `cosine_sim()` and `cosine_recos()` functions earlier to find the top 5 most similar books to a book of your choice.
- ii. Do the recommendations make sense? Why does cosine similarity “know” about similarity of book descriptions, without being told anything about plot or genre?

Question 2) Correlation is at the heart of many data analytic methods so let's explore it further.

In our `compstatslib` package, you will find an `interactive_regression()` function that runs a simulation of simple linear regression. You can click to add data points to the plotting area and see a corresponding regression line (hitting ESC will stop the simulation). You will also see three numbers: regression intercept – where the regression line crosses the y-axis; regression coefficient – the slope of x on y; correlation - correlation of x and y. This is a *simple* regression because it is only one x-var versus one y-var.

For each of the scenarios below, create the described set of points in the simulation. You might have to create each scenario a few times to get a general sense of them. Visuals of the scenarios are shown on the next page.

- a. Scenario A: Create a horizontal set of random points, with a relatively narrow but flat distribution.
 - i. What *raw slope* of x and y would you *generally* expect?
 - ii. What is the correlation of x and y that you would *generally* expect?
- b. Scenario B: Create a random set of points to fill the entire plotting area, along both x-axis and y-axis.
 - i. What *raw slope* of the x and y would you *generally* expect?
 - ii. What is the correlation of x and y that you would *generally* expect?
- c. Scenario C: Create a diagonal set of random points trending upwards at 45 degrees
 - i. What *raw slope* of the x and y would you *generally* expect? (note that x, y have the same scale)
 - ii. What is the correlation of x and y that you would *generally* expect?
- d. Scenario D: Create a diagonal set of random trending downwards at 45 degrees
 - i. What *raw slope* of the x and y would you *generally* expect? (note that x, y have the same scale)
 - ii. What is the correlation of x and y that you would *generally* expect?



- Apart from any of the above scenarios, find another pattern of data points with no correlation ($r \approx 0$).
(can you create a pattern that visually suggests a strong relationship but produces $r \approx 0$?)
- Apart from any of the above scenarios, find another pattern of data points with perfect correlation ($r \approx 1$).
(can you find a scenario where the pattern visually suggests a non-perfect relationship?)
- Let's see how correlation relates to simple regression, by simulating any *linear relationship* you wish:
 - Run the simulation and record the points you create: `pts <- interactive_regression()`
(simulate either a positive or negative relationship)
 - Use the `lm()` function to estimate the *regression intercept and slope* of `pts` to ensure they are the same as the values reported in the simulation plot: `summary( lm( pts$y ~ pts$x ) )`
 - Calculate the correlation of `x` and `y` using `cor(pts)` — is it the same as reported in the plot?
 - Now, *standardize* the values of *both* `x` and `y` from `pts` and re-estimate the regression slope
 - What is the relationship between *correlation* and the *standardized simple-regression estimates*?

Submitting Your Work

Report: I encourage you to start using [RMarkdown](#) or [Quarto](#) to produce your report (we will soon require it).
This week, you may use AI tools so long as it is not to write all the code. Please state: “I have largely solved the problems without the use of any AI agent or tool”. I do encourage using AI to understand any results or produce your report (Quarto or RMarkdown). You may add to your statement: “However, I have used AI to [...]” where you may tell us in what ways AI has helped you—please be as specific as you can.

Anonymous: Do NOT put your name on the assignment; Only put your student ID at the top of each page

Credit Help: If anyone has helped you greatly, mention it and add their student ID at the top of your assignment and explain very briefly (7 words or less) how they helped.

For each question:

- Show the question number you are solving
- Provide a brief (< 1 line) description of the problem
- Show your code (format as code — do not use a screenshot of code)
- Show *relevant* results (see Canvas link on how to insert R plot graphics into MS Word)

Submit PDF: Please upload your report on Canvas as a PDF file.